SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1-CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO5	**Assessment:**	ASSESSMENT TOOL1-Constraint-Based Problem Solving
Weightage:	40%	**Total Marks:**	25
CO5 Statement:	*Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.*		
Student Name:	D. Snyana Bai	**Reg.No.:**	192571067
Department:	CS & BIOSCIENCE	**Date:**	

ANNEXURE A

Constraint-Based Problem Solving

1.Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2.Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3.Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4.Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5.Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 µs
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I D. Sayana Rao (1925 21069) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

D. Sayana Rao

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1	2	1.5	2	1	0.5
Q2	2	1.5	2	1	0.5
Q3	2	2	2.5	1.5	1
Q4	2.5	2	2.5	1	1
Q5	2.5	2.5	2	1	1

41

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25% 2.5	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25% 2.5	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25% 2.5	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15% 1.5	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10% 1	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Q1. Construct an I/O Communication Mechanism for a Real-Time Embedded System

Problem Understanding

The system contains multiple sensors generating interrupts every 1 ms. The CPU utilization must remain below 15%, vectored interrupt handling must be used, and an optimal I/O method must be selected.

Constraint Analysis

A real-time embedded system requires immediate response to sensor events. Since interrupts occur every millisecond, the processor may become overloaded if interrupt servicing is inefficient. Therefore:

- CPU utilization must be minimized.
- Interrupt handling latency must be reduced.
- Sensor data should be accessed quickly.
- Fast identification of interrupt sources is required.

Solution Development

Proposed Architecture

The system uses:

1. Multiple Sensors
2. Interrupt Controller
3. Vectored Interrupt Table
4. Memory-Mapped I/O Registers
5. CPU
6. Data Buffer

Design Approach

Memory-Mapped I/O is selected because device registers can be accessed using normal memory instructions, reducing execution overhead and improving response time.

When a sensor generates an interrupt:

1. Interrupt Controller receives the request.
2. Vectored Interrupt Controller provides the ISR address directly.
3. CPU jumps directly to the ISR.
4. Sensor data is read from Memory-Mapped Registers.
5. Data is stored in a buffer for processing.

Summary Table

Constraint	Solution
CPU Utilization < 15%	Use efficient ISR and buffering
Interrupt every 1 ms	Vectored interrupt handling
Low latency required	Memory-Mapped I/O
Multiple sensors	Interrupt controller with priority logic

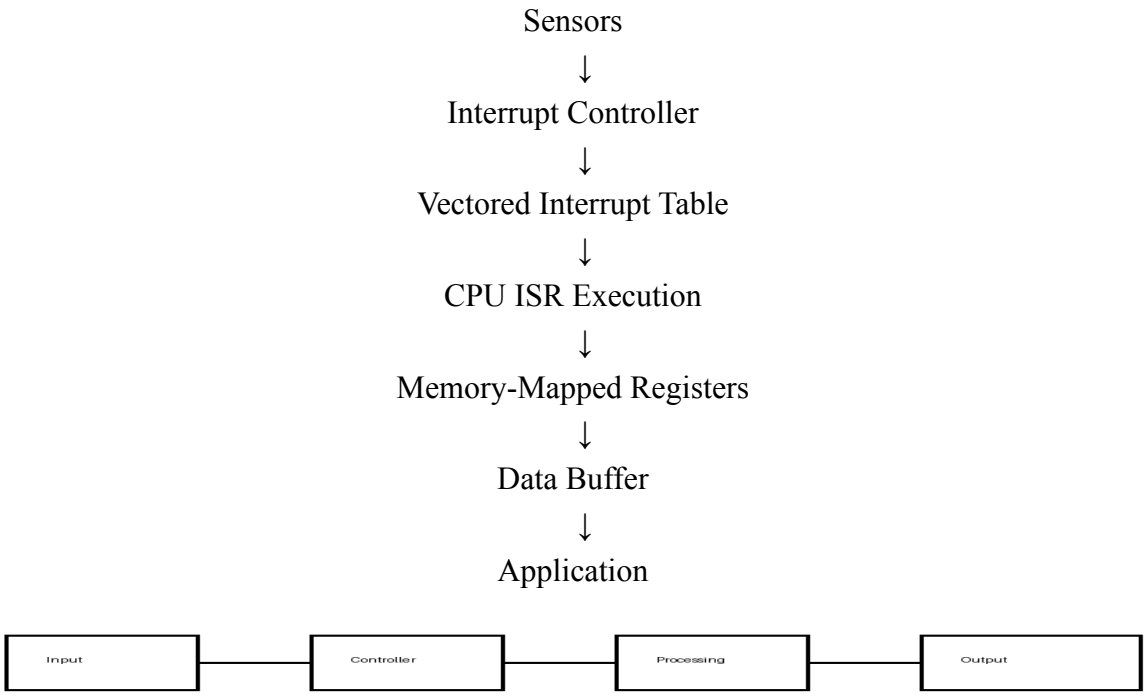
Application of Concepts

- Memory-Mapped I/O
- Vectored Interrupts
- Interrupt Priority Management
- Real-Time Processing

Justification

Memory-Mapped I/O reduces access time compared with I/O-mapped techniques. Vectored interrupts eliminate polling overhead by directly locating the ISR address. As a result, CPU utilization remains below 15% while maintaining reliable real-time operation.

Flow Chart



Q2. Design an I/O Subsystem for High-Speed Storage

Problem Understanding

The storage subsystem must achieve throughput greater than 3 GB/s using NVMe over PCIe. CPU involvement in bulk transfer should be eliminated through DMA. Interrupt notification must be used.

Constraint Analysis

The major challenge is achieving very high throughput without burdening the CPU.
Key requirements:

- Throughput > 3 GB/s
- NVMe protocol support
- DMA-based transfer
- Efficient completion notification

Solution Development

Proposed Architecture

The architecture consists of:

- NVMe SSD
- PCIe Interface
- DMA Controller
- Main Memory
- Interrupt Controller
- CPU

Working Principle

1. CPU issues an NVMe read/write command.
2. NVMe controller initiates DMA transfer.
3. Data moves directly between SSD and RAM.
4. CPU remains free for other tasks.
5. Completion interrupt is generated after transfer.

Summary Table

Constraint	Solution
>3 GB/s throughput	NVMe over PCIe Gen4
CPU should not transfer data	DMA Controller
Fast communication	PCIe lanes
Completion notification	Interrupt mechanism

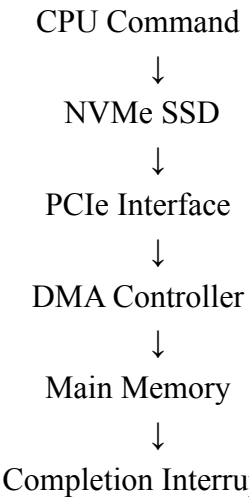
Application of Concepts

- NVMe Protocol
- PCIe Bus Architecture
- Direct Memory Access
- Interrupt-Based Notification

Justification

NVMe uses PCIe's parallel lanes and supports multiple command queues. DMA removes CPU involvement in data movement. This architecture easily exceeds 3 GB/s throughput while maintaining system efficiency.

Flow Chart



↓

CPU Notification

Q3. Construct a Multi-Device Communication Architecture

Problem Understanding

The system contains six USB peripherals and two Thunderbolt devices. Interrupt starvation must be avoided and fair bus arbitration must be ensured.

Constraint Analysis

Multiple devices competing for bus access may create:

- Bus contention
- Interrupt starvation
- Unfair resource allocation

Therefore arbitration and interrupt prioritization are necessary.

Solution Development

Proposed Architecture

Components:

- USB Hub
- Thunderbolt Controller
- Bus Arbiter
- Interrupt Controller
- CPU

Working Process

1. USB and Thunderbolt devices send requests.
2. Bus Arbiter uses Round Robin scheduling.
3. Interrupt Controller assigns priorities.
4. Hardware interrupts serve urgent tasks.
5. Software interrupts handle secondary tasks.

Summary Table

Constraint	Solution
6 USB devices	USB Hub
2 Thunderbolt devices	Thunderbolt Controller
Fair arbitration	Round Robin Arbiter
Avoid starvation	Priority-based interrupt scheduling

Application of Concepts

- Bus Arbitration
- Hardware Interrupts
- Software Interrupts
- Round Robin Scheduling

Justification

Round Robin arbitration guarantees equal bus access opportunities. Hardware interrupts handle time-critical events, while software interrupts process less critical tasks, preventing starvation.

Flow Chart

USB Devices + Thunderbolt Devices
↓
Bus Arbiter
↓
Interrupt Controller
↓
CPU
↓
Resource Allocation

Q4. Design a Data Acquisition System

Problem Understanding

The system continuously receives data at 500 MB/s. CPU intervention must be minimized, DMA and interrupt-driven I/O must be compared, and memory-mapped registers must be used.

Constraint Analysis

At 500 MB/s, interrupt-driven transfer would generate excessive CPU overhead. DMA is necessary for efficient continuous acquisition.

Proposed Architecture

Components:

- High-Speed Sensor
- Memory-Mapped Registers
- DMA Controller
- RAM Buffer

Working Process

1. Sensor generates data stream.
2. DMA transfers data directly to memory.
3. CPU processes only completed blocks.
4. Completion interrupt signals transfer completion.

Comparison Table

Feature	DMA Transfer	Interrupt-Driven I/O
CPU Usage	Very Low	High
Transfer Speed	Very High	Moderate
Suitable for 500 MB/s	Yes	No
Efficiency	Excellent	Poor

Application of Concepts

- DMA Controller
- Memory-Mapped I/O

- Interrupt Processing

Justification

DMA significantly reduces CPU workload and supports continuous high-speed streaming. Interrupt-driven I/O would generate thousands of interrupts, increasing processor overhead and reducing performance.

Flow Chart

Sensor Stream
↓
Memory-Mapped Registers
↓
DMA Controller
↓
RAM Buffer
↓
Completion Interrupt
↓
CPU Processing

Q5. Construct an Interrupt Handling Mechanism

Problem Understanding

The system must support nested interrupts, achieve latency below 10 μs, use vectored interrupts for high-priority devices, and software interrupts for background processing.

Constraint Analysis

Critical devices require immediate servicing while background activities should not interfere with urgent tasks.

Requirements:

- Nested interrupt support
- Less than 10 μs latency
- Priority management
- Background task handling

Proposed Architecture

Components:

- Priority Interrupt Controller
- Vectored Interrupt Table
- CPU
- Software Interrupt Handler

Working Process

1. High-priority interrupt occurs.
2. Vectored controller provides ISR address.
3. CPU services interrupt immediately.
4. If a higher-priority interrupt arrives, nesting occurs.
5. Background jobs execute through software interrupts.

Summary Table

Constraint	Solution
Nested interrupts	Priority stacking
Latency <10 µs	Vectored interrupts
High-priority servicing	Hardware interrupts
Background tasks	Software interrupts

Application of Concepts

- Nested Interrupts
- Vectored Interrupts
- Hardware Interrupts
- Software Interrupts
- Interrupt Priority Levels

Justification

Vectored interrupts provide the fastest interrupt response by directly locating the ISR. Nested interrupt handling ensures urgent requests are processed immediately, helping the system maintain latency below 10 µs.

Flow Chart